WQD7009 Big Data Applications and Analytics

Individual Assignment

Name: Faiz Yah Bin Abdul Hanan Yah
Student ID: 24087980
Date: 23rd November 2025

Table of Contents

1 Introduction.....3
1.1 Significance of Analysis..... 3
1.2 Analysis Approach..... 3
1.3 Dataset Introduction.....3

2 Hadoop and Spark Operation and Performance Comparative Analysis..... 6
2.1 Operations.....6
2.2.1 Operation 1: Unique Value Count.....7
2.2.2 Operation 2: Cardinality.....8
2.2 Performance Comparison..... 8
2.3 Explanation for Performance Difference.....11

3 Analysis of Data Insights..... 11

Appendix..... 13
Script for Operation 1: Unique Value Count..... 13
Script for Operation 2: Cardinality..... 14

1 Introduction

Sustainable Development Goals (SDGs) has played and is continuing to play a critical role in this day and age where sustainability is of utmost priority. SDGs is all-encompassing, consisting of 17 different goals and 169 targets which include poverty, hunger and food security, health, quality education, gender equality, water and clean sanitation, economic growth and urban city, responsible production and consumption, climate change and biodiversity. Amongst these diverse 17 goals, this dataset and analysis scopes down on Goal #13 which is about climate action, specifically studying the **landfill gas (LFG) energy projects across the United States**.

1.1 Significance of Analysis

Landfill gas which is primarily made up of methane and carbon dioxide is produced naturally as organic waste decomposes in landfills. Methane is a dangerous greenhouse gas with a global warming potential that is much greater than that of carbon dioxide, underscoring the gravity of the situation. By successfully capturing and converting these gas into energy, it assists in generating renewable electricity while simultaneously preventing significant greenhouse gas emissions which would otherwise damage the climate.

1.2 Analysis Approach

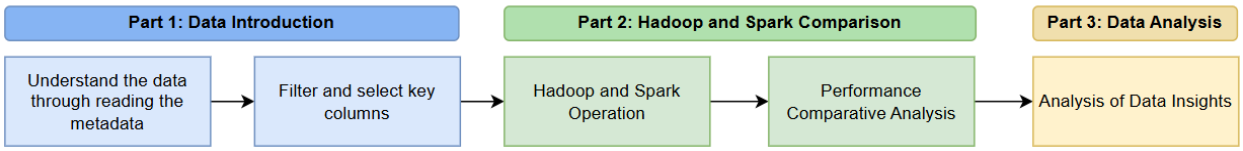


Diagram 1: Flowchart of Analysis

A strategic approach is deployed to effectively analyse and obtain meaningful insights. Firstly, it is crucial to read through the metadata and description of each of the 32 columns to understand better the context and purpose of the dataset. Then, we proceed by trimming down and filtering to select only the key columns that will be beneficial for our analysis. Once we arrive at the desired columns, we can proceed with carrying out the operations in both Hadoop and Spark. We continue with the very principal objective of comparing the performance between Hadoop and Spark. The outcome from this comparative analysis allows us to tangibly witness the difference in performance for both frameworks. Lastly, we conduct a thorough analysis of the data to obtain meaningful insights and propose several recommendations.

1.3 Dataset Introduction

The dataset is named *US Landfills & Renewable Energy Dataset* which provides an extensive view of landfill operations, landfill gas management, and energy production projects across the

United States. The data allows for a comprehensive understanding on how waste management practices are able to convert unwanted waste into renewable energy and subsequently reduce the emission of greenhouse gas. The data was originally obtained from Kaggle, which after further investigation reveals that the very origin of the data was sourced from United States Environmental Protection Agency (EPA) under their Landfill Gas Energy Project Data.

The dataset has 31 columns and 873 rows of records. The information of each the columns are explained as below:

No.	Data Field	Description	Data Type
1	GHGRP ID	Identifier for GHGRP	Numerical
2	Landfill ID	Identifier for landfill	Numerical
3	Landfill Name	Landfill name	Categorical
4	State	State landfill is located in	Categorical
5	City	City landfill is located in	Categorical
6	County	County landfill is located in	Categorical
7	Ownership Type	Indicates if landfill is publicly owned, privately owned, or co-owned by public and private entities	Categorical
8	Landfill Owner Organization(s)	Organization that owns the landfill	
9	Waste in Place (tons)	Waste-in-place at the landfill in short tons	Numerical
10	Waste in Place Year	Year corresponding to the waste-in-place at the landfill (YYYY)	Timestamp
11	LFG Collection System In Place?	Is there a landfill gas collection system in place? 'Yes' for an active GCCS; 'No' if landfill has no gas collection system, is passively venting/flaring, or has perimeter gas wells.	Categorical
12	LFG Collected (mmscfd)	Amount of landfill gas being collected in million standard cubic feet per day	Numerical
13	LFG Flared (mmscfd)	Amount of landfill gas flared (if project is operational, amount of landfill gas flared in back-up flare(s)) in million standard cubic feet per day	Numerical
14	Project ID	Unique identifying number combination for "parent" project and "expansion" number	Numerical
15	Current Project Status	Current project status (Operational, Construction, Planned, Shutdown, Candidate, Future Potential, Low Potential, Unknown)	Categorical
16	Project Name	Specific project name (if applicable)	Categorical
17	Project Start Date	Date project or expansion became operational (MM/DD/YYYY)	Timestamp
18	Project Finish Date	Date project or expansion ended operation (MM/DD/YYYY)	Timestamp
19	Project Type Category	Category of LFG energy project type (Electricity, Direct, Renewable Natural Gas)	Categorical

20	LFG Energy Project Type	Specific type of LFG energy project technology	Categorical
21	RNG Delivery Method	Method for delivery of the renewable natural gas (Pipeline Injection, Local Use, Unknown)	Categorical
22	LFG Use Details	Additional details or specifications about how the landfill gas is utilised in the project	Categorical
23	Actual MW Generation	Actual megawatt electrical power currently being generated by the project	Numerical
24	Rated MW Capacity	Maximum rated megawatt capacity that the equipment is designed to produce	Numerical
25	LFG Flow to Project (mmscfd)	Amount of landfill gas flowing to the energy project in million standard cubic feet per day	Numerical
26	End User(s)	Name of end user organization	Categorical
27	Project Developer(s)	Name of the organization that developed the landfill gas energy project	Categorical
28	Project Owner(s)	Name of the organization that owns and operates the energy project	Categorical
29	Product Supplier(s)	Name of the company that supplied the equipment or technology for the project	Categorical
30	Self-Developed	Indicates whether the project was developed by the landfill owner themselves (Yes/No)	Categorical
31	Current Year Emission Reductions (MMTCO2e/yr) - Direct	Annual direct greenhouse gas emission reductions achieved by the project, measured in million metric tons of carbon dioxide equivalent per year (MMTCO2e/yr)	Numerical

Table 1 : Original Data Fields of Dataset

For the context of this analysis, only certain columns are required. Here, the approach is to remove columns that mainly act as identifiers, names or descriptions. After filtering for only the necessary columns, we went from 31 columns to 14 columns, of which the selected column are as below:

No.	Data Field
1	Landfill Name
2	State
3	City
4	County

5	Ownership Type
6	Waste in Place (tons)
7	Waste in Place Year
8	LFG Collected (mmscfd)
9	LFG Flared (mmscfd)
10	Current Project Status
11	LFG Energy Project Type
12	RNG Delivery Method
13	Actual MW Generation
14	Current Year Emission Reductions (MMTCO2e/yr) - Direct

Table 2 : Filtered Data Fields of Dataset

2 Hadoop and Spark Operation and Performance Comparative Analysis

2.1 Operations

There are two operations that will be carried for both Hadoop and Spark, which are detailed as follow:

1. **Unique Value Count:** Identify how many times each unique value appears
2. **Cardinality:** Identify how many unique values exists

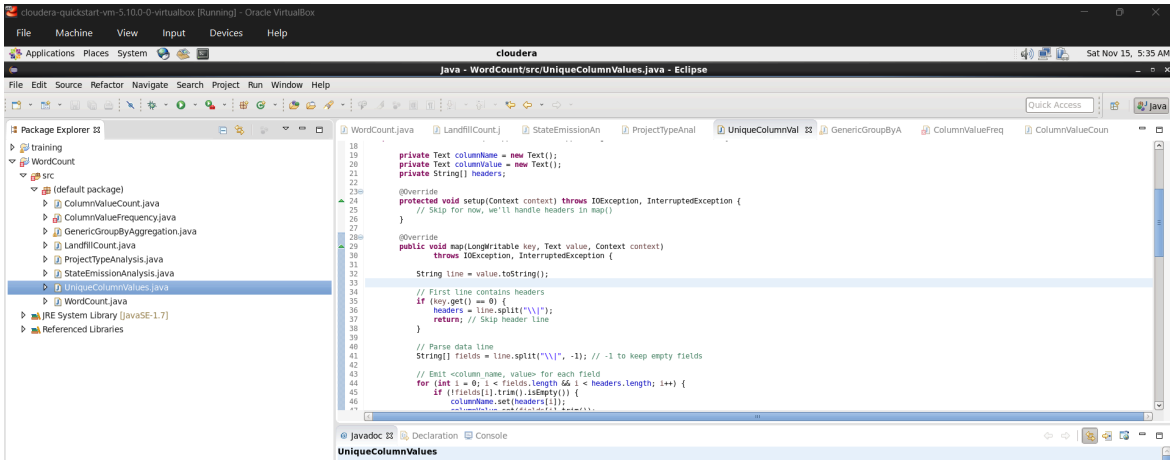


Diagram 2: Hadoop Java Class Script in Eclipse

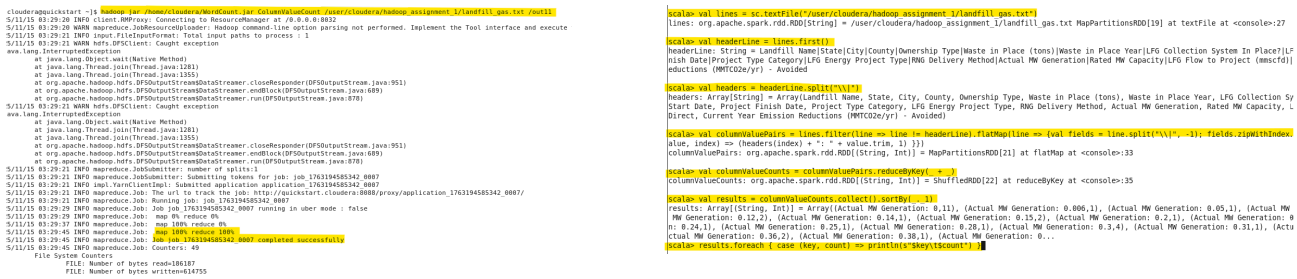


Diagram 3: Hadoop (Left) compared to Spark (Right) — Code Execution

For Hadoop, a Java Project is first created in Eclipse followed by the Java class script to carry out the two operations, as shown in Diagram 2, after which only the operations can be carried out on the terminal . While for Spark, the process is less tedious since we can use spark-shell on the terminal. This implies that the respective syntax and functions can be directly executed on the terminal.

2.2.1 Operation 1: Unique Value Count

This operation is similar to that of WordCount, in which we identify how many times each unique value occurs in the columns. Identifying the count of unique values allows us to understand the distribution of the data better, whether there are certain values that dominate, or certain values that are insignificant in quantity. Each unique value for each of the columns are printed out one by one on the terminal for both Hadoop and Spark. The diagram below represents a snapshot of the output. Taking *State: FL* as an example, it means that the state Florida of the USA appears 30 times throughout the 873 rows of data. Evidently, we can observe that the output for both Hadoop and Spark are similar in nature, indicating a success.

```
Rated MW Capacity: 9.2  3
Rated MW Capacity: 9.3  8
Rated MW Capacity: 9.6  13
Rated MW Capacity: 9.9  1
Self-Developed: No      562
Self-Developed: Unknown 43
Self-Developed: Yes     268
State: AK       2
State: AL       5
State: AR       3
State: AZ       1
State: CA       87
State: CO       4
State: CT       4
State: DE       7
State: FL       30
State: GA       25
State: IA       8
State: ID       5
State: IL       29
```

```
Rated MW Capacity: 9.15 1
Rated MW Capacity: 9.2  3
Rated MW Capacity: 9.3  8
Rated MW Capacity: 9.6  13
Rated MW Capacity: 9.9  1
Self-Developed: No      562
Self-Developed: Unknown 43
Self-Developed: Yes     268
State: AK       2
State: AL       5
State: AR       3
State: AZ       1
State: CA       87
State: CO       4
State: CT       4
State: DE       7
State: FL       30
State: GA       25
State: IA       8
State: ID       5
State: IL       29
State: IN       32
State: KS       8
State: KY       12
State: LA       10
```

Diagram 4: Hadoop (Left) compared to Spark (Right) — Unique Value Count

2.2.2 Operation 2: Cardinality

Cardinality operation aims to identify how many unique values are present for each of the columns. Once we identify the cardinality, it helps in understanding whether certain columns are binary in nature, or the majority of the values are unique.

```
========================================
Column: State
Unique Count: 49
----------------------------------------
VT, RI, NY, GA, NV, TN, CA, ME, OK, VA, MI, OH
Waste in Place (tons)
========================================
Column: Waste in Place (tons)
Unique Count: 475
----------------------------------------
13,802,719, 9,546,388, 2,626,797, 22,938,750,
,297, 20,788,471, 16,544,910, 4,034,215, 4,005
5,718,750, 1,769,760, 5,736,457, 12,798,269, 5
Waste in Place Year
========================================
Column: Waste in Place Year
Unique Count: 31
----------------------------------------
2008, 2009, 2006, 2007, 2004, 2005, 2002, 2003
```

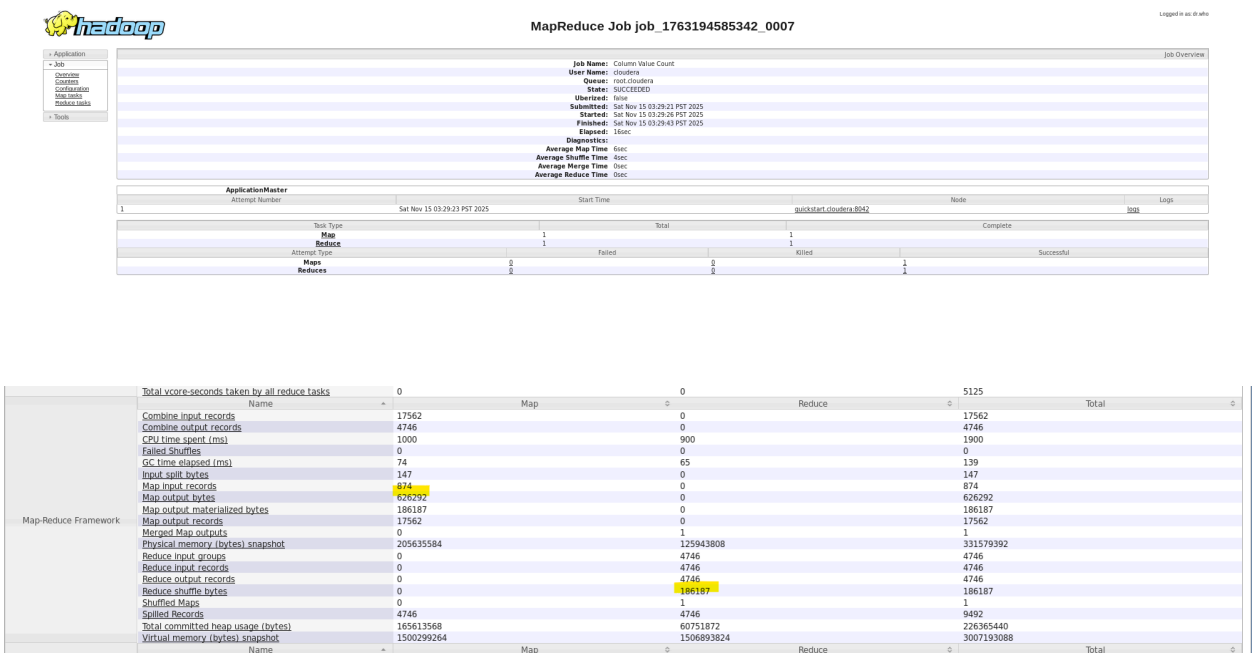
```
========================================
Column: State
Unique Count: 49
----------------------------------------
MD, NY, CO, SC, ME, PR, WA, DE, WV, NE, IA, TX, RI, AZ, NM, NV, MI, CT, GA, VT,

========================================
Column: Landfill Name
Unique Count: 486
----------------------------------------
Tuscarora Long-Term Regional Landfill, Richland Creek Road Landfill, Glen's Sani
c., Greenwood Farms Landfill, Clover Flat Landfill, Ottawa County LF, Iredell Co
n SLF, Johnson Canyon SLF, Lycoming County Landfill, IESI Timber Ridge Landfill,
Facility, Meadow Branch Landfill, Columbus County, ARS, Forward Landfill, City o
 Frank R. Bowerman SLF, City of Bristol Virginia Integrated Solid Waste Manageme
., Rapid City Landfill, Fighting Creek Farm Landfill, Eastern Sanitary Landfill

========================================
Column: County
Unique Count: 337
----------------------------------------
Tazewell, Craven, Weld, Santa Barbara, Surry, Dorchester, Arapahoe, Mahoning, An
n, Boone, Denton, Napa, Fajardo, DeKalb, Sauk, McLeod, Marin, Marion, Dougherty,
```

Diagram 5: Hadoop (Left) compared to Spark (Right) — Cardinality

Referring to Diagram 4, it is observable that the output for both Hadoop and Spark are not entirely the same in terms of unique value sequence, however are similar in terms of cardinality. For example, for Hadoop on the left, it calculates 49 unique states which has the similar calculation for Spark of 49 unique states. This once again signifies the success of the operation.

2.2 Performance Comparison

Although both Hadoop and Spark are able achieve the same outcome implementing two different frameworks, the next important question would then be which one is more effective in terms of performance, which is also the crux of this project. To identify their performance, one method is to access their job report on their respective sites. For Hadoop, the access site is *quickstart.cloudera:19888/jobhistory/app*, while Spark would be l*ocalhost:4040/jobs/*. The user interface of these both frameworks can be referred to at Diagram 5 and Diagram 6.



MapReduce Job job_1763194585342_0007

Job Name:	Column Value Count
User Name:	cloudera
Queue:	root.cloudera
State:	SUCCEEDED
Uberized:	false
Submitted:	Sat Nov 15 03:29:21 PST 2025
Started:	Sat Nov 15 03:29:26 PST 2025
Finished:	Sat Nov 15 03:29:43 PST 2025
Elapsed:	16sec
Diagnostics:	
Average Map Time	6sec
Average Shuffle Time	4sec
Average Merge Time	0sec
Average Reduce Time	0sec

ApplicationMaster

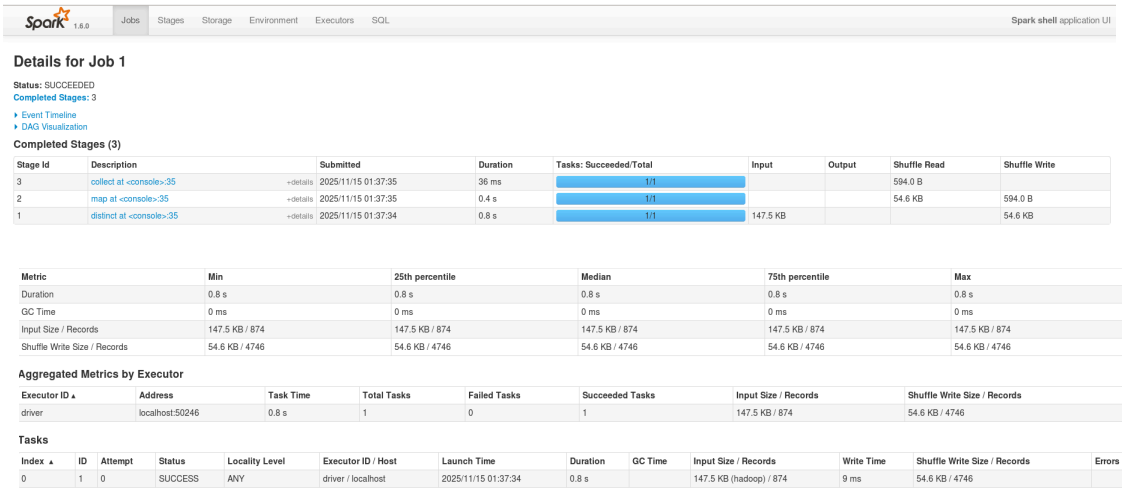
Attempt Number	Start Time	Node	Logs
1	Sat Nov 15 03:29:23 PST 2025	quickstart.cloudera:8042	logs

Task Type	Total	Complete
Map	1	1
Reduce	1	1

Attempt Type	Failed	Killed	Successful
Maps	0	0	1
Reduces	0	0	1

	Name	Map	Reduce	Total
	Total vcore-seconds taken by all reduce tasks	0	0	5125
Map-Reduce Framework	Combine input records	17562	0	17562
	Combine output records	4746	0	4746
	CPU time spent (ms)	1000	900	1900
	Failed Shuffles	0	0	0
	GC time elapsed (ms)	74	65	139
	Input split bytes	147	0	147
	Map input records	874	0	874
	Map output bytes	626292	0	626292
	Map output materialized bytes	186187	0	186187
	Map output records	17562	0	17562
	Merged Map outputs	0	1	1
	Physical memory (bytes) snapshot	205635584	125943808	331579392
	Reduce input groups	0	4746	4746
	Reduce input records	0	4746	4746
	Reduce output records	0	4746	4746
	Reduce shuffle bytes	0	186187	186187
	Shuffled Maps	0	1	1
	Spilled Records	4746	4746	9492
	Total committed heap usage (bytes)	165613568	60751872	226365440
	Virtual memory (bytes) snapshot	1500299264	1506893824	3007193088
	Name	Map	Reduce	Total

Diagram 6: Hadoop Job User Interface for Operation 1



Spark 1.6.0 — Jobs | Stages | Storage | Environment | Executors | SQL — Spark shell application UI

Details for Job 1

Status: SUCCEEDED
Completed Stages: 3

Event Timeline
DAG Visualization

Completed Stages (3)

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
3	collect at <console>:35 +details	2025/11/15 01:37:35	36 ms	1/1			594.0 B	
2	map at <console>:35 +details	2025/11/15 01:37:35	0.4 s	1/1			54.6 KB	594.0 B
1	distinct at <console>:35 +details	2025/11/15 01:37:34	0.8 s	1/1	147.5 KB			54.6 KB

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	0.8 s	0.8 s	0.8 s	0.8 s	0.8 s
GC Time	0 ms	0 ms	0 ms	0 ms	0 ms
Input Size / Records	147.5 KB / 874	147.5 KB / 874	147.5 KB / 874	147.5 KB / 874	147.5 KB / 874
Shuffle Write Size / Records	54.6 KB / 4746	54.6 KB / 4746	54.6 KB / 4746	54.6 KB / 4746	54.6 KB / 4746

Aggregated Metrics by Executor

Executor ID ▲	Address	Task Time	Total Tasks	Failed Tasks	Succeeded Tasks	Input Size / Records	Shuffle Write Size / Records
driver	localhost:50246	0.8 s	1	0	1	147.5 KB / 874	54.6 KB / 4746

Tasks

Index ▲	ID	Attempt	Status	Locality Level	Executor ID / Host	Launch Time	Duration	GC Time	Input Size / Records	Write Time	Shuffle Write Size / Records	Errors
0	1	0	SUCCESS	ANY	driver / localhost	2025/11/15 01:37:34	0.8 s		147.5 KB (hadoop) / 874	9 ms	54.6 KB / 4746	

Diagram 7: Spark Job User Interface for Operation 1

Dissecting firstly Hadoop's Operation 1 based on Diagram 5, we obtain that the time taken to process is 16 seconds, while the *main output bytes* is 626KB and the *reduce shuffle bytes* is 186KB. Comparing this metrics with that of Spark's, we observe based on Diagram 6 that it takes around 1.2s to process, with both Shuffle Read and Shuffle Write being both 54.6KB.

Task	Time Taken to Process		Shuffle Read		Shuffle Write	
	Hadoop	Spark	Hadoop	Spark	Hadoop	Spark
Unique Value Count	16 seconds	1.2s	626KB	52KB	186KB	52KB
Cardinality	15 seconds	0.9s	538KB	55KB	574KB	55KB

Table 3: Hadoop and Spark Performance Comparison

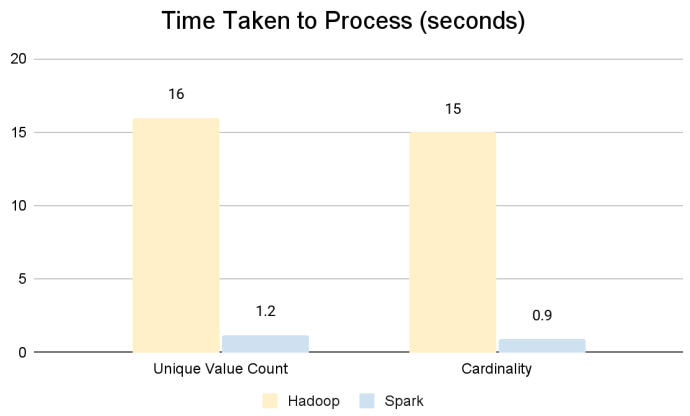


Diagram 8: Execution Time in Second

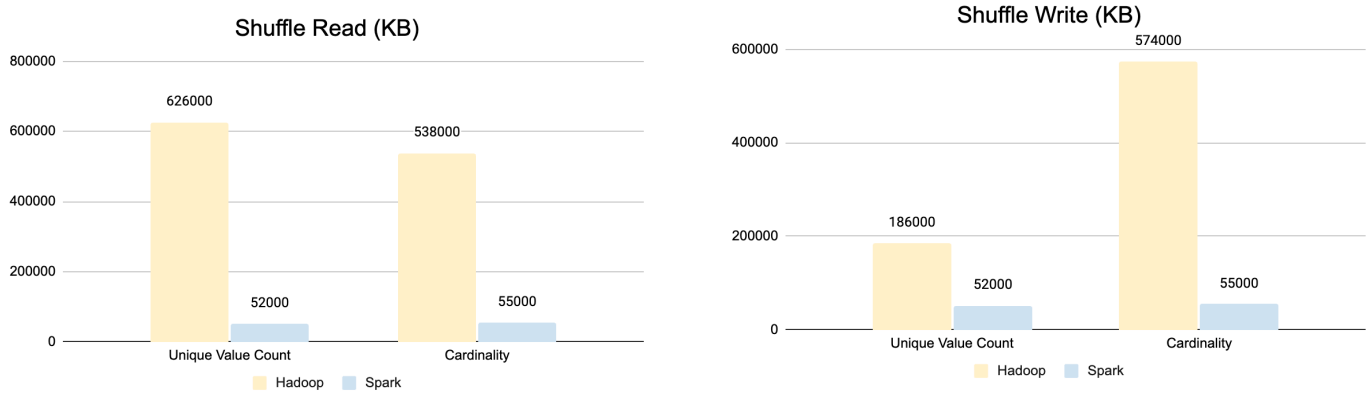


Diagram 9: Memory Consumption in KB (Read and Write)

Tabulating (Table 3) and visualizing the outcome (Diagram 8 and 9) provides a much better view on the magnitude of difference between the two frameworks. In terms of execution time for both operations, Spark takes roughly only 1s while Hadoop requires an average of 15.5 seconds, which means Spark is 15 times faster. Looking at the memory consumption when reading and writing, Hadoop demands at minimum 186KB to a maximum of 626KB of memory, while Spark

requires at minimum 52KB to a maximum of 55KB. Firstly, this shows that Hadoop demands higher memory consumption, up to 12 times more memory than Spark. Secondly, it also signifies that Hadoop is less consistent as compared to Spark since the difference between the minimum and maximum for Hadoop is almost 3.3 times while Spark is relatively consistent at 52-55KB. This establishes the fact that Spark wins at all fronts in terms of both speed and memory consumption, making it the more superior framework in this context.

2.3 Explanation for Performance Difference

The substantial difference observed between Hadoop and Spark can be explained from their architectural design difference. Hadoop MapReduce relies on disk-based processing, resulting in slower I/O process, while Spark leverages in-memory processing through Resilient Distributed Dataset (RDD), which reduces disk I/O operations. In addition, Hadoop is required to process each MapReduce job independently while Spark utilises Directed Acyclic Graph (DAG) execution engine with has higher optimisation.

3 Analysis of Data Insights

After tabulating the output from the terminal for both Operation 1 and Operation 2, we obtain a table as below:

No.	Data Field	**Operation 1:** Top 5 Values in Terms of Frequency Count (Count)	**Operation 2:** Cardinality (%)
1	Landfill Name	Lanchester LF (8), ..., Monterey Peninsula SLF (8)	55.67%
2	State	CA (87), MI (75), PA (69), NY (53), VA (45)	5.61%
3	City	Marina (8), Narvon (8), Rochester (8), Dallas (7), East Brunswick (7)	50.97%
4	County	Los Angeles (17), Jefferson (12), Lancaster (11), Orange (11), San Diego (11)	38.60%
5	Ownership Type	Private (489), Public (384)	0.23%
6	Waste in Place (tons)	1,000,000 (8) ; 13,569,947 (8) .... ; 39,427,783 (8)	54.52%
7	Waste in Place Year	2021 (668), 2013 (19), 1996 (15), 2007 (11), 1997 (9)	3.67%
8	LFG Collected (mmscfd)	2.349 (8), 2.92 (8), 10.91 (8), 1.52 (7), 0.581 (6)	53.04%
9	LFG Flared (mmscfd)	0 (54), 0.023 (8), 0.306 (8), 0.004 (7), 0.64 (7)	25.89%

10	Current Project Status	Operational (697), Shutdown (164), Planned (9), Construction (2), Future Potential (1)	0.57%
11	LFG Energy Project Type	Reciprocating Engine (490), Vehicle Fuel (118), Cogeneration (67), Boiler (49), Gas Turbine (48)	1.83%
12	RNG Delivery Method	(Blanks) (748), Pipeline Injection (110), Local Use (15)	0.23%
13	Actual MW Generation	(Blanks) (250), 1,6 (17), 3.2 (16), 0.8 (15), 0 (11)	36.08%
14	Current Year Emission Reductions (MMTCO2e/yr) - Direct	0 (202), 0.081 (16), 0.0405 (10), 0.0759 (8), 0.1012 (8)	47.88%

A few insights can be made from observing Operation 1 which identifies the top 5 values and their corresponding count. We notice that the states with the most landfills are California, Michigan, Pennsylvania which are top big states by population, indicating that there might be a correlation between big states and amount of landfills. Shifting our attention to *Ownership Type*, a larger majority of landfills are run privately instead of publicly. An interesting finding is from the *Waste in Place Year,* we can see a huge increase in recent years, with 668 of the landfills being in the year 2021 compared to only 15 in 1996, which might translate to the generation of more waste leading to more landfills. A positive outlook can be derived from the Current *Project Status* column which indicates that a majority of landfills are still in operation and there are even upcoming ones being planned.

Checking on Operation 2 that focuses on cardinality, we observe that the percentage does reflect what is expected. Numerical data that is continuous in nature such as *Waste in Place (tons)*, *LFG Collected (mmscfd)* and *Actual MW Generation* have higher cardinality. On the flip side, columns with a set number of categorical values such as *Ownership Type* (either Private or Public) and *Current Project Status* (with 5 options) have much lower cardinality.

Considering all the analysis and insights, it can be seen that there is still room for improvement, for instance, the *Current Year Emission Reductions* shows that one-fourth of landfills are still at zero. A few recommendations can be proposed in hope of improving this initiative in achieving a better outlook in the long run. Firstly, the *LFG Energy Project Type* seems to be concentrated on Reciprocating Engine, a recommendation would then be to diversify and explore other potential types. Secondly, it would be beneficial to address the non-operational sites, as in to investigate the 164 landfills that are currently shutdown as an opportunity to restart or repurpose the site. Thirdly, we can learn from states that are successful and replicate their best practices on sub-optimal states.

Appendix

Script for Operation 1: Unique Value Count

a. Hadoop Map Reduce

```java
import org.apache.hadoop.conf.Configuration;

public class ColumnValueCount {

    public static class CountMapper extends Mapper<LongWritable, Text, Text, IntWritable> {

        private Text columnValuePair = new Text();
        private final static IntWritable one = new IntWritable(1);
        private String[] headers;

        @Override
        public void map(LongWritable key, Text value, Context context)
                throws IOException, InterruptedException {

            String line = value.toString();

            // First line - get headers
            if (key.get() == 0) {
                headers = line.split("\\|");
                return;
            }

            // Skip if headers not loaded
            if (headers == null) {
                return;
            }

            // Parse data line
            String[] fields = line.split("\\|", -1);

            // Emit each column:value pair
            for (int i = 0; i < fields.length && i < headers.length; i++) {
                String fieldValue = fields[i].trim();
                if (!fieldValue.isEmpty()) {
                    String key_out = headers[i] + ": " + fieldValue;
                    columnValuePair.set(key_out);
                    context.write(columnValuePair, one);
                }
            }
        }
```

```java
public static class CountReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    @Override
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {

        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }

        context.write(key, new IntWritable(sum));
    }
}
```

Hadoop Map function (left) and Reduce function (right)

b. Spark

```scala
val lines = sc.textFile("/user/cloudera/hadoop_assignment_1/landfill_gas.txt")

val headerLine = lines.first()

val headers = headerLine.split("\\|")

val columnValuePairs = lines.filter(line => line != headerLine).flatMap(line => {val fields = line.split("\\|", -1);
fields.zipWithIndex.filter { case (value, index) => value.trim.nonEmpty && index < headers.length }.map { case (value,
index) => (headers(index), value.trim) }}).distinct()

val groupedByColumn = columnValuePairs.groupByKey()

val results = groupedByColumn.map { case (columnName, values) => val valueList = values.toList; val uniqueCount =
valueList.size; val result = new StringBuilder; result.append("\n========================================\n"); result.append
(s"Column: $columnName\n"); result.append(s"Unique Count: $uniqueCount\n"); result.append
("----------------------------------------\n"); val displayValues = valueList.take(50); result.append(displayValues.mkString
(", ")); if (uniqueCount > 50) {result.append(", ...")}; (columnName, result.toString) }

results.saveAsTextFile("/user/cloudera/hadoop_assignment_1/output_unique_spark_shell")
```

Spark written with Scala on Terminal

Script for Operation 2: Cardinality

c. Hadoop Map Reduce

```java
import org.apache.hadoop.conf.Configuration;

public class UniqueColumnValues {

    public static class UniqueMapper extends Mapper<LongWritable, Text, Text, Text> {

        private Text columnName = new Text();
        private Text columnValue = new Text();
        private String[] headers;

        @Override
        protected void setup(Context context) throws IOException, InterruptedException {
            // Skip for now, we'll handle headers in map()
        }

        @Override
        public void map(LongWritable key, Text value, Context context)
                throws IOException, InterruptedException {

            String line = value.toString();

            // First line contains headers
            if (key.get() == 0) {
                headers = line.split("\\|");
                return; // Skip header line
            }

            // Parse data line
            String[] fields = line.split("\\|", -1); // -1 to keep empty fields

            // Emit <column_name, value> for each field
            for (int i = 0; i < fields.length && i < headers.length; i++) {
                if (!fields[i].trim().isEmpty()) {
                    columnName.set(headers[i]);
                    columnValue.set(fields[i].trim());
                    context.write(columnName, columnValue);
                }
            }
        }
    }
```

```java
public static class UniqueReducer extends Reducer<Text, Text, Text, Text> {

    @Override
    public void reduce(Text key, Iterable<Text> values, Context context)
            throws IOException, InterruptedException {

        // Use HashSet to collect unique values
        Set<String> uniqueValues = new HashSet<>();

        for (Text val : values) {
            uniqueValues.add(val.toString());
        }

        // Output format: Column: [count] unique values
        StringBuilder result = new StringBuilder();
        result.append("\n======================================\n");
        result.append("Column: ").append(key.toString()).append("\n");
        result.append("Unique Count: ").append(uniqueValues.size()).append("\n");
        result.append("--------------------------------------\n");

        int count = 0;
        for (String uniqueVal : uniqueValues) {
            if (count > 0) result.append(", ");
            result.append(uniqueVal);
            count++;

            // Limit output to avoid very long lines
            if (count >= 50) {
                result.append(", ...");
                break;
            }
        }

        context.write(key, new Text(result.toString()));
    }
}
```

Hadoop Map function (left) and Reduce function (right)

d. Spark

```scala
val lines = sc.textFile("/user/cloudera/hadoop_assignment_1/landfill_gas.txt")

val headerLine = lines.first()

val headers = headerLine.split("\\|")

val columnValuePairs = lines.filter(_ != headerLine).flatMap(line => {val fields = line.split("\\|",
-1); fields.zipWithIndex.filter { case (v, i) => v.trim.nonEmpty && i < headers.length }.map { case
(v, i) => (headers(i), v.trim) }})

val uniqueCounts = columnValuePairs.distinct().map { case (col, value) => (col, 1) }.reduceByKey(_ +
_).collect().sortBy(_._1)

results.saveAsTextFile("/user/cloudera/hadoop_assignment_1/output_unique_spark_shell")
```

Spark written with Scala on Terminal